SMART REFRIGERATOR AI PROJECT

BY SHYAM, JENSEN, CHANTELLE, AND AHANTYA

PROBLEM STATEMENT

- Households struggle with food waste, and managing meals.
- Our Fridge AI project will help customer with food item tracking, expiry tracking, and recipe generation.
- Reduced food waste and increased efficiency in time
- Result → sustainability, health awareness, and lower inflation.

DATASET & PREPROCESSING: Dataset Description

	Unnamed: 0	title	ingredients	directions	link	source	NER
0	0	No-Bake Nut Cookies	["1 c. firmly packed brown sugar", "1/2 c. eva...	["In a heavy 2-quart saucepan, mix brown sugar...	www.cookbooks.com/Recipe-Details.aspx?id=44874	Gathered	["brown sugar", "milk", "vanilla", "nuts", "bu...
1	1	Jewell Ball'S Chicken	["1 small jar chipped beef, cut up", "4 boned ...	["Place chipped beef on bottom of baking dish....	www.cookbooks.com/Recipe-Details.aspx?id=699419	Gathered	["beef", "chicken breasts", "cream of mushroom...
2	2	Creamy Corn	["2 (16 oz.) pkg. frozen corn", "1 (8 oz.) pkg...	["In a slow cooker, combine all ingredients. C...	www.cookbooks.com/Recipe-Details.aspx?id=10570	Gathered	["frozen corn", "cream cheese", "butter", "gar...
3	3	Chicken Funny	["1 large whole chicken", "2 (10 1/2 oz.) cans...	["Boil and debone chicken.", "Put bite size pi...	www.cookbooks.com/Recipe-Details.aspx?id=897570	Gathered	["chicken", "chicken gravy", "cream of mushroo...
4	4	Reeses Cups(Candy)	["1 c. peanut butter", "3/4 c. graham cracker ...	["Combine first four ingredients and press in ...	www.cookbooks.com/Recipe-Details.aspx?id=659239	Gathered	["peanut butter", "graham cracker crumbs", "bu...

The dataset used in this project is sourced from the **RecipeNLG** dataset on Kaggle: https://www.kaggle.com/datasets/paultimothymooney/recipenlg.

This dataset, named **RecipeNLG_dataset.csv** (2.14GB), contains **2,231,142 recipes** with comprehensive information.

Title (string)	Name of the recipe
Ingredients (string)	List of ingredients
Directions (string)	Cooking instructions
Link (string)	Source URL
Source (string)	Original content source
NER (string)	Named entity recognition labels
# (int)	Identifier for the recipe

DATASET & PREPROCESSING: Dataset Preparation

Cleaning: Removed rows with missing ingredients/direction, to ensure clean text inputs.

Tokenization: Used the GPT-2 tokenizer to convert raw text into token ID that suitable for model input.

Standardize format for Prompt Response: Standardized the data into prompt response format <INPUT_START> ingredients1 <NEXT_INPUT> ingredients2 <INPUT_END> (e.g., "Ingredients: egg, flour, sugar → Recipes: [recipe text]") to align the model's training.

Dataset Split: Divided the dataset into training and test sets.

Model Saving: The fine-tuned model was saved as gpt2-recipes-manual, ensure we can load the trained weights for GPT2LMHeadModel in the transformer library.

DATASET & PREPROCESSING: Filtering Criteria for Recipe Selection

- Predefined list includes: milk, egg, tomato, spinach, apple, etc.
- Focus on ingredients commonly found in the fridge
- Recipes must have 2 to ingredients
- At least one ingredient must be a common fridge item
- Incomplete or low quality records are removed

```
# Define common fridge items
fridge_items = {
    "milk", "cheese", "cream cheese", "butter", "yogurt", "sour cream", "egg",
    "tomato", "onion", "lettuce", "spinach", "cabbage", "zucchini", "carrot",
    "garlic", "cucumber", "pepper", "celery", "apple", "grapes", "corn", "mushroom"
}

# Fridge inventory filter
def has_fridge_item(ingredients):
    match_count = sum(item.lower() in fridge_items for item in ingredients)
    return 2 <= len(ingredients) <= 10 and match_count >= 1

# Apply filter
df_subset = df[df["ingredients"].apply(has_fridge_item)].sample(frac=0.1,
random_state=42)

df_subset = df_subset.dropna(subset=["text"])
```

MODEL TRAINING: GPT-2 Transformer-Based Language Model

Why GPT-2?

- Trained on diverse text, ensuring comprehension language understanding and semantics.
- Autoregressive architecture ideal for logical, and well structured text like recipe instructions.
- Easily adapted to domain-specific dataset for recipe generation
- Using Transformers library GPT2LMHeadModel to simplifies implementation with fine tuning

MODEL TRAINING: Set Up & Process

Training Setup

- Model: GPT-2 `GPT2LMHeadModel.from_pretrained("gpt2")`
- Tokenizer: GPT-2 with padding token `tokenizer.pad_token = tokenizer.eos_token`
- Device: CUDA GPU or CPU
- Optimizer: AdamW, learning rate: `lr=5e-5`

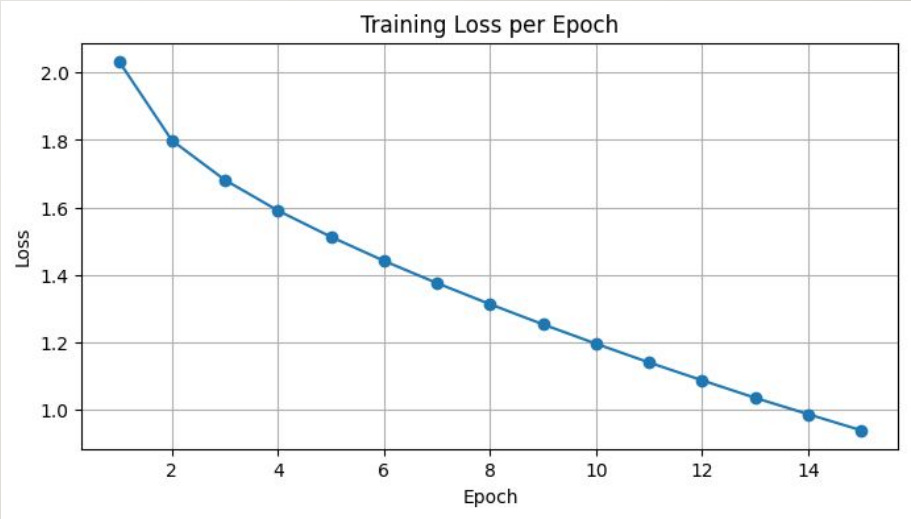
Training Loop

- Epoch: 15
- Dataset: `RecipeNLG_dataset` from `filtered_recipes_gpt.txt`
- Batch size: 2
- Process:
  - Forward pass with input IDs and attention mask
  - Compute loss, backpropagate, and update weights
  - Track avg loss per epoch for analysis

```
optimizer = torch.optim.AdamW(model.parameters(),
lr=5e-5)
model.train()

epochs = 15
epoch_losses = []
for epoch in range(epochs):
    total_loss = 0
    for batch in tqdm(loader):
        inputs = batch["input_ids"].to(device)
        attention_mask =
batch["attention_mask"].to(device)

        outputs = model(input_ids=inputs,
attention_mask=attention_mask, labels=inputs)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        total_loss += loss.item()

    avg_loss = total_loss / len(loader)
    epoch_losses.append(avg_loss)
```

MODEL TRAINING: Training Loss Result

- **Epochs:** 15
- **Initial loss:** ~2.03
- **Final loss:** ~0.94
- **Trend:** Steady decline, indicating effective model learning and convergence.



OUTPUT RESULTS: Output Configuration

Parameter	Value	Description
max_length	120	Set to keep the output concise while ensure enough tokens for instructions
do_sample	True	Allows variation in recipe generation
top_k	50	Limit sampling to most likely words
top_p	0.92	Cumulative probability = 92%, allows diversity without too many randomness
temperature	0.65	Control random outputs, maintain recipe reasonably structured
repetition_penalty	1.3	Increased penalty, improves output diversity

Prompt Format:

```
Ingredients: egg, flour, sugar → Recipe:
```

Evaluation Mode:

```
model.eval()
```

CHALLENGES (LIKE DATA POISONING) & SOLUTIONS

- During testing we found issue with recipe generator not returning valid recipes but instead we got food related URLs
- To fix this we have a post generation function that check for suspicious result to guardrail against this type of behaviour
- Like this function that can be expanded to check for future invalid input:

```
def is_suspicious(text):
    return re.search(r'https?://', text, re.IGNORECASE) is not None
```

- Inside the recipe_generator() function we retry up to 3 times for valid result

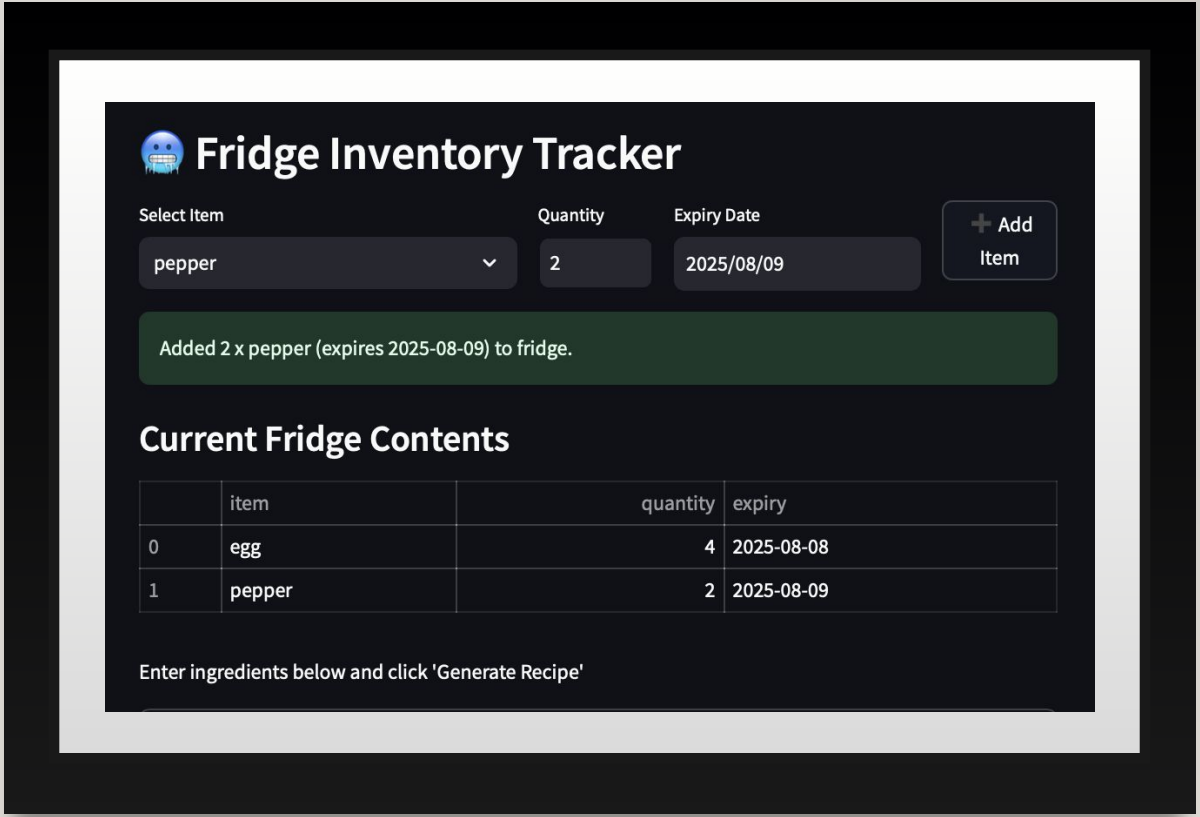
```
if not is_suspicious(result):
    return result
else:
    st.warning(f"Suspicious content detected. Retrying... (Attempt {attempt}/{max_retries})")
```

- Why? → Retry with the **same prompt** the model will generate a **different** output

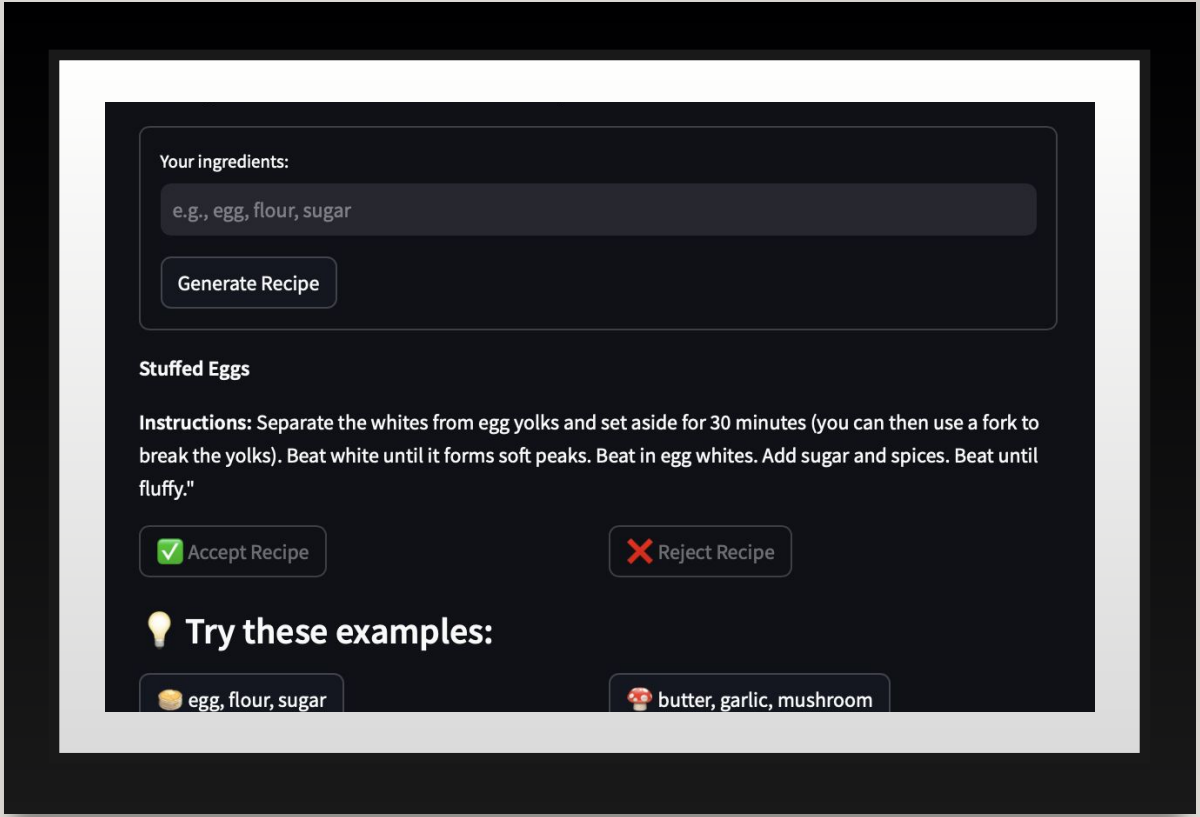
INTEGRATION IN THE APPLICATION STACK

1. **User Input** via Streamlit – User enters available ingredients.
2. **Inventory Tracker** – Tracks items and expiration.
3. **GPT-2 Model** – Generates new recipes based on user input.
4. **User Feedback** – Accept/reject logic used for removing used items from inventory

FRONT END
Inventory



FRONT END Generator



ARCHITECTURE OVERVIEW - Systems and Constraints

Goal: Local **GPT-2** recipe generator + **RAG** for factual grounding + **Streamlit** UI.
Key constraint: *No LLM API endpoint calls*; architecture prioritized over raw accuracy.
UI: Streamlit for ingredient entry, results, and feedback.
Model: Fine-tuned **GPT-2** (decoder-only Transformer) for next-token recipe generation.
Retrieval: LangChain to inject nutrition/substitution facts (reduces hallucinations).
Storage: Inventory/expiry in **DuckDB / SQLite / Parquet** (local).
Dataset: RecipeNLG (Kaggle); 2,231,142 rows with title/ingredients/directions/link/source/NER/id.
Filtering focus: Common fridge items; 2–10 ingredients; drop incomplete/low-quality rows.

ARCHITECTURE OVERVIEW - Data Flow, Runtime & Reliability

Logical Flow:

Streamlit UI → Inventory DB → GPT-2 (fine-tuned) → Recipe Output + Accept/Reject

Training & runtime

- **Hardware:** Colab (T4/L4) and local GPUs (Mac/Windows).
- **Core params:** batch **2**, epochs **15**, LR **1e-5** (report) / **5e-5** (training deck).
- **Artifacts:** Saved weights **gpt2-recipes-manual** for reloadable inference.

Reliability / guardrails

- Post-gen URL check + up to **3 retries** to block bad outputs; practical fix for “URL” failure mode.

ARCHITECTURE RESEARCH - GPT-2 Internals & Design Rationale

Transformer (decoder-only) blocks: token + positional embeddings → multi-head self-attention → FFN → residuals + layer norm → softmax.
Objective: causal language modeling (cross-entropy next-token prediction).
Why GPT-2 for this project:

- Open-source fine-tuning; no API dependency.
- Coherent, stepwise text (recipe instructions) with good compute/quality trade-off.
- Easy integration via Transformers (GPT2LMHeadModel).

ARCHITECTURE RESEARCH - Fine-Tuning, Inference, Integration & Evaluation

Preprocess & format: clean rows; normalize units; GPT-2 tokenize; **prompt→response** format (“Ingredients: … → Recipe: …”).
Training loop: AdamW, LR **5e-5** (deck) / **1e-5** (report); **15 epochs**; track epoch loss; save model.
Observed convergence: loss **~2.03** → **0.94** over 15 epochs.
Generation config (inference): `max_length=120`, `do_sample=True`, `top_k=50`, `top_p=0.92`, `temperature=0.65`, `repetition_penalty=1.3`.
Future RAG integration: LangChain retrieves nutrition/substitutions; grounds outputs and reduces hallucinations.
Evaluation: User acceptance rate (accept/reject).